STA 3402 – Machine Learning & Neural Networks

Lecture 2 (Day 2) – Dimension Reduction: Principal Component Analysis (PCA)

Lecturer: Dr. Samir, Department of Statistics, University of Colombo

Full study notes from the lecture recording, the whiteboard, the R code and the PCA slides (simple English)

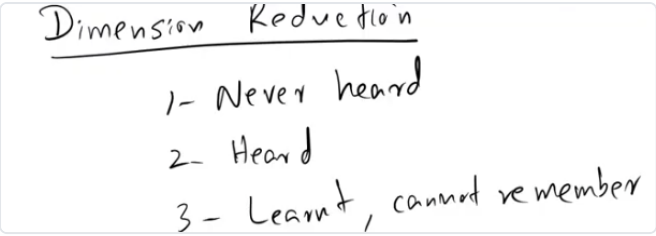
Contents

1. Class notes (assignment upload, holidays)
2. What is dimension reduction?
3. The main idea of PCA
4. The steps of PCA
5. How many PCs? – variance explained and the scree (elbow) plot
6. PCA in R – `prcomp()` and `princomp()`
7. R practical: PCA on mtcars (step by step, with the mistakes to avoid)
8. Writing down a principal component (loadings)
9. PCA is an intermediate step
10. Application: image compression (the cat example)
11. The theory of PCA – a graphical explanation
12. PCA with 3 or more variables, and when PCA does not work
13. Summary, homework and revision questions

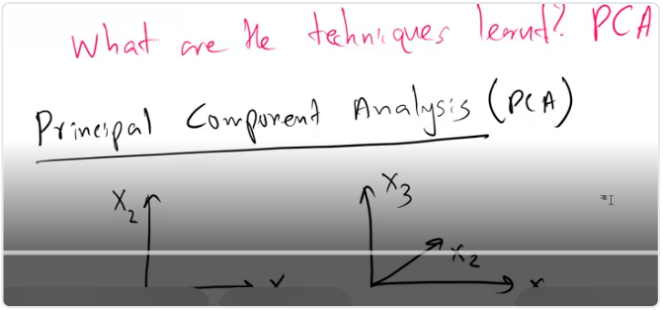
1. Class notes

- Upload your assignment files to the given link before the deadline (the lecturer said "before 10.00", please check the exact time in the WhatsApp group).
- **Christmas holidays:** start from the next day evening and end the next Monday. Lectures start again on Monday, so the next class is on **Monday the 29th**.
- A **lab sheet** on PCA will be given for the lab session next week. The slides will be shared.

2. What is dimension reduction?



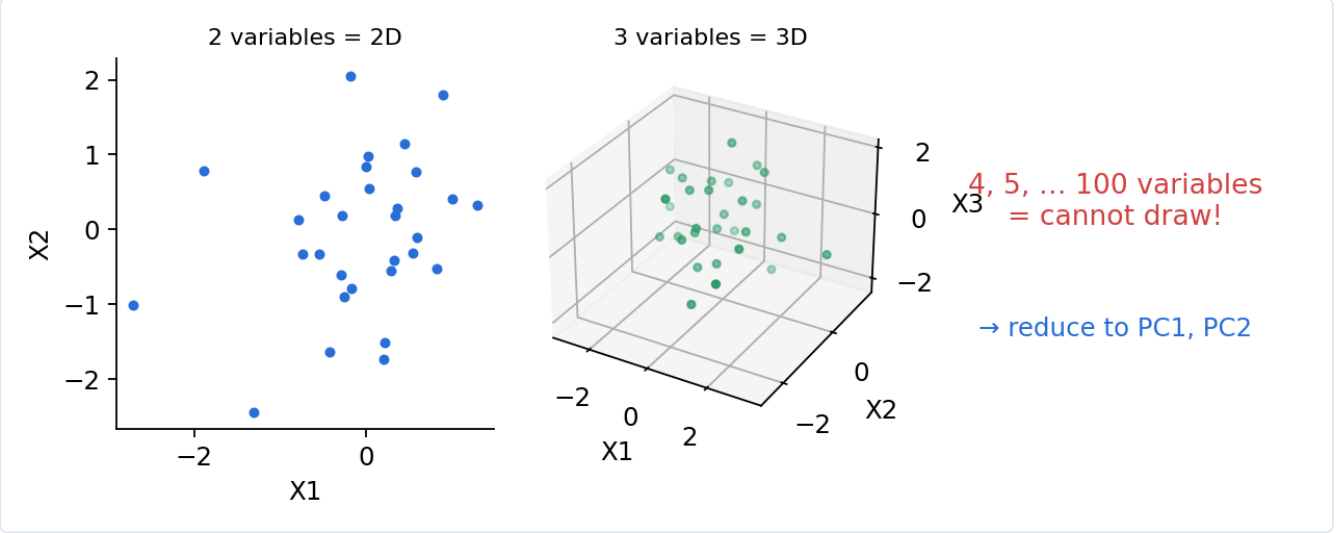
Class survey: 1 = never heard, 2 = heard, 3 = learnt but cannot remember



Techniques learnt: PCA. Two variables → 2D, three → 3D

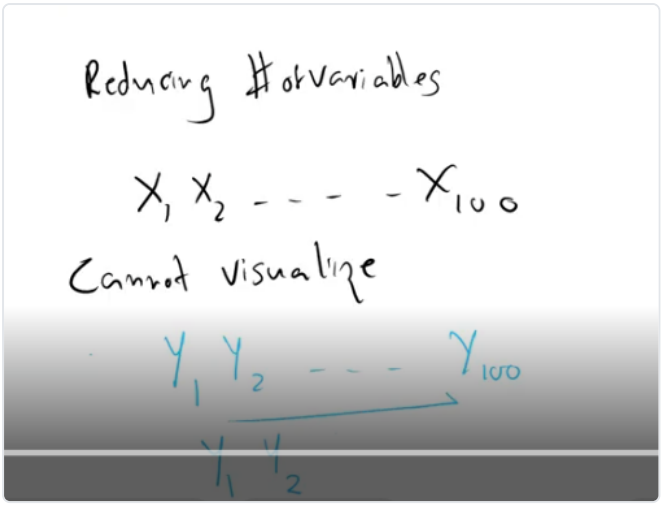
The lecturer asked what we know about dimension reduction. Some students had learnt **Principal Component Analysis (PCA)** in an optional multivariate statistics course. So PCA is taught **from scratch** here.

- **Dimension = number of variables.** Two variables (X1, X2) → a 2-D graph. Three variables (X1, X2, X3) → a 3-D graph. With more variables we have more dimensions.
- **Dimension reduction** = reducing the number of variables.
- Important: it does **not** mean we simply throw away some variables. We keep all of them in the calculation, but we **create a new system of variables** that carries most of the information in fewer variables.

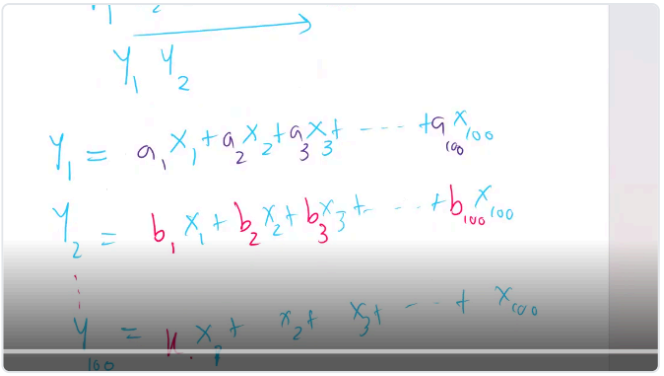


We can draw 2 or 3 dimensions, but not 100

3. The main idea of PCA



Whiteboard: reducing the number of variables



Whiteboard: each new variable is a linear combination

Suppose we have **100 variables** X_1, X_2, ..., X_{100}. We **cannot visualize** them (we can only see up to 3 dimensions; 2 is ideal).

So we create a **new set of 100 variables** Y_1, Y_2, ..., Y_{100} such that:

- **Y_1** explains the **largest** amount of variation in the data,
- **Y_2** explains the **second largest** amount, and so on, in **descending order**.

Each new variable is a **linear combination** of **all** the original variables:

$$Y_1 = a_1X_1 + a_2X_2 + a_3X_3 + \ldots + a_{100}X_{100}$$

$$Y_2 = b_1X_1 + b_2X_2 + b_3X_3 + \ldots + b_{100}X_{100}$$

$$\vdots$$

$$Y_{100} = k_1X_1 + k_2X_2 + \ldots + k_{100}X_{100}$$

(a, b, ..., k are constants.) These new variables are called **principal components (PCs)**. 100 original variables give 100 PCs.

Because Y_1 and Y_2 hold most of the variation, we can **plot only Y_1 and Y_2**. Now we can see the data, maybe find **clusters**, and then, for example, fit a **separate regression for each cluster**. This is very useful in areas like **bioinformatics**, where there are thousands of variables (genes).

4. The steps of PCA

Class exercise: "give me the main steps in PCA". Student answers: standardize, get the covariance matrix, calculate eigenvalues and eigenvectors, select eigenvalues, form the feature vector, transform. The lecturer's full list:

> **Rule 0 – data type**
> PCA works **only for quantitative (numeric) data**. It does not work for categorical data. Remove categorical variables first.

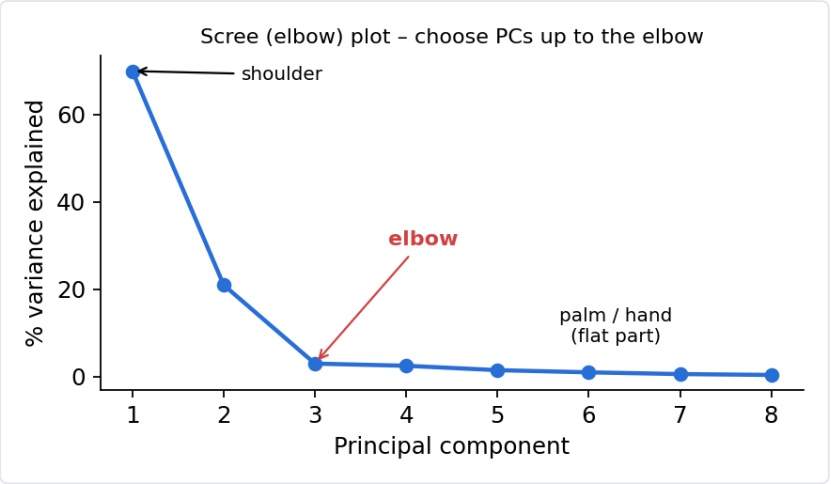
1. **Standardize** the data. Variables can be in different units with different variation, so we bring them to the same scale (subtract the mean, divide by the standard deviation).
2. Find the **variance–covariance matrix** of the standardized data.
3. Find the **eigenvalues** and **eigenvectors** of this matrix.
4. Order them: the **largest eigenvalue** and its eigenvector → PC1; the next largest → PC2; and so on.
5. The eigenvectors give the **loadings** (the coefficients a_1, a_2, ...).
6. Write down the **new (transformed) data set** – the PC scores.
7. Decide **how many PCs** are needed, using the variance explained and the **scree plot**.

5. How many PCs? – variance explained and the scree plot

If there are 100 variables, there are 100 PCs. For each PC we can calculate the **variation explained**. Example:

PC	% variation explained	Cumulative %
PC1	70%	70%
PC2	21%	70 + 21 = 91%
PC3	3%	94%
...	...	...

There is no point in keeping all 100 PCs – our goal is to **reduce** the number of variables. To decide how many to keep we use the **scree plot** (also called the **elbow plot**): for each PC we plot the variance explained.



Think of the line as an **arm**: the high part at the start is the **shoulder**, the flat part at the end is the **palm**, and the bend is the **elbow**. Theory says: **keep the PCs up to the elbow point**. Here the elbow is at PC2 or PC3 → use 2 or 3 PCs.

6. PCA in R – prcomp() and princomp()

Class answer: two functions can be used.

	`prcomp()`	`princomp()`
Package	**stats** (default, no install/load needed)	**stats** (default, no install/load needed)
Method	Singular value decomposition (SVD) of the data – more accurate numerically	Eigen decomposition of the covariance (or correlation) matrix
Loadings stored in	`$rotation`	`$loadings`
Scores stored in	`$x`	`$scores`
Standardize option	`center = TRUE, scale. = TRUE`	`cor = TRUE`
Note	Works even with more variables than rows. Usually preferred.	Needs more rows than variables. Uses divisor n instead of $n - 1$.

You can check the package with the help page: `?prcomp` shows "prcomp {stats}", and `?princomp` shows "princomp {stats}".

7. R practical: PCA on mtcars

Exercise: (1) load mtcars; (2) carry out PCA; (3) use `plot()` to get the scree plot; (4) how many PCs do you recommend?

The general pattern the lecturer gave: if your data is called `data`, then `PCAout = prcomp(data)` and `plot(PCAout)` gives the scree plot.

Step 1 – Load the data

```
# STA 3402 – Day 2
# 1. load the mtcars dataset
data(mtcars)
dim(mtcars)          # 32 rows, 11 columns

# 2. carry out the PCA
prcomp()
princomp()
# do we need any additional packages?
?prcomp              # prcomp {stats} -> comes with the stats package
?princomp            # princomp {stats}
```

The first attempt – and why it is WRONG

```
PCAout = prcomp(mtcars)
summary(PCAout)      # PC1 ≈ 92.7%, PC2 ≈ 7.2%, rest very small
plot(PCAout)         # elbow at 1 or 2 PCs
```

This gives PC1 ≈ 92% and PC2 ≈ 7%, which looks nice, but **the answer is wrong**. Why? (A student answered correctly.)

1. The **categorical variables were not removed**. PCA works only for quantitative data.

2. The data were **not standardized**. `disp` and `hp` have very large numbers (hundreds), so they dominate PC1 just because of their units.

Step 2 – Find the categorical variables

```
# remove the categorical variable
str(mtcars)          # all columns show as "num" – not helpful here!
?mtcars              # read the variable description
```

`str()` shows every column as numeric, because the data set stores everything as numbers. But some of them are really **categorical (qualitative)**. From the help page:

Variable	Meaning	Type
mpg	Miles per gallon	Quantitative ✔
cyl	Number of cylinders	Quantitative (integer) ✔
disp	Displacement	Quantitative ✔
hp	Gross horsepower	Quantitative ✔
drat	Rear axle ratio	Quantitative ✔
wt	Weight	Quantitative ✔
qsec	Quarter-mile time	Quantitative ✔
vs	Engine type (0 = V-shaped, 1 = straight)	**Categorical ✘**
am	Transmission (0 = automatic, 1 = manual)	**Categorical ✘**
gear	Number of forward gears	Can be taken as quantitative ✔
carb	Number of carburetors	Can be taken as quantitative ✔

So we must remove **vs** and **am** (columns 8 and 9).

Step 3 – Remove vs and am (work on a copy!)

```
## create a copy of the dataset
data = mtcars

# option 1: remove the vs and am variables (NOT recommended)
data$vs = NULL
data$am = NULL

# option 2 (recommended): extract only the quantitative columns
data_quants = data[,-c(8,9)]
data_quants = data[,c(1:7,10,11)]
dim(data_quants)     # 32 rows, 9 columns
```

- **Always make a copy** of a built-in data set (`data = mtcars`). Do not change the original, otherwise next time you will not get the full data set.
- `data$vs = NULL` deletes the column **permanently** from that data frame – **not recommended**.
- `data[ , -c(8,9)]` means: all rows (row part empty), all columns **except** 8 and 9. `data[ , c(1:7,10,11)]` picks columns 1 to 7, 10 and 11. Both give the same result. This is

recommended, because the original stays unchanged and you may need the categorical variables later in the analysis.

> **Careful when running the code**
> In the R file, both options are written one after the other. If you run option 1 first, `data` already has only 9 columns, so `data[,-c(8,9)]` would then remove **gear and carb** by mistake. Use **only one** option: run option 2 on a fresh copy (`data = mtcars`).

A wrong idea: sapply(data, is.numeric)

```
# this is not going to work
dim(data)
Numaric_data = data[, sapply(data, is.numeric)]
dim(Numaric_data)    # nothing removed!
```

- `sapply(data, is.numeric)` applies `is.numeric()` to every column and returns TRUE/FALSE. Only TRUE columns are kept.
- But **vs and am are stored as numbers (0 and 1)**, so `is.numeric()` gives TRUE for them too, and **nothing is removed**.
- It would only work if the categorical variables were stored properly as **factors**.
- Lesson: AI tools sometimes suggest code like this without knowing what is really in the data. **Think and check** (e.g. with `dim()`).

Step 4 – Standardize the data

```
# standardize the data
data_quants$mpg
scale(data_quants$mpg)          # one variable

data_standadized = scale(data_quants)   # all columns at once
```

- `scale(x, center = TRUE, scale = TRUE)` subtracts the mean (center) and divides by the standard deviation (scale). Set one to FALSE if you want only subtracting or only dividing.
- For example, mpg 21.0 (first car) becomes 0.1509.
- `scale(data_quants)` does this for every column. The output also stores the means and standard deviations used (as attributes).

Another way (a student's answer): let `prcomp()` standardize inside the function:

```
pca_result = prcomp(data_quants, center = TRUE, scale. = TRUE)
```

This gives the same result as `prcomp(scale(data_quants))`.

Step 5 – Run PCA and read the summary

```
prcomp_out = prcomp(data_standadized)
summary(prcomp_out)
```

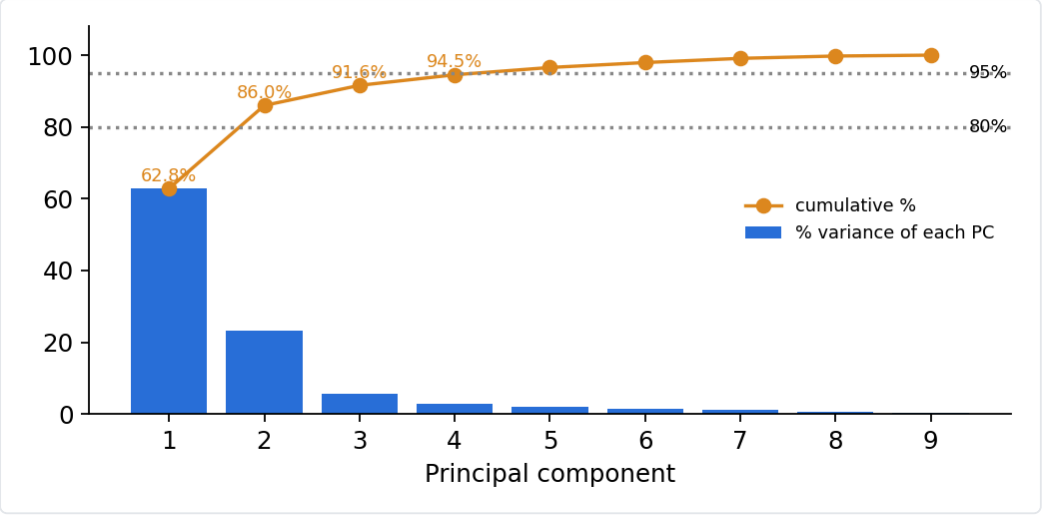
The lecturer's mistake in class (and why it matters)
In class, the lecturer first standardized `data` (which still had vs and am) instead of `data_quants`. The result was PC1 = 60.08%, PC2 = 24.1%, PC3 = 5.7%, PC4 = 2.45% (cumulative 60.08%, 84.17%, 89.87%, 92.32%). Later, while looking at the loadings, he saw **vs and am** in the output and realised the mistake – they should not be there. Lesson: **be vigilant, do not just accept whatever output you get**. Think whether it makes sense.

Correct output (standardized, 9 quantitative variables):

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8	PC9
Standard deviation	2.378	1.443	0.710	0.515	0.428	0.352	0.324	0.242	0.149
Proportion of variance	0.628	0.231	0.056	0.029	0.020	0.014	0.012	0.007	0.003
Cumulative proportion	0.628	0.860	**0.916**	0.945	0.966	0.979	0.991	0.998	1.000

How to read it: PC1 explains 62.8% of the variation. PC2 explains 23.1%. Together PC1 + PC2 explain 86.0% (the **cumulative** value adds them). PC1 + PC2 + PC3 explain **91.58%**.

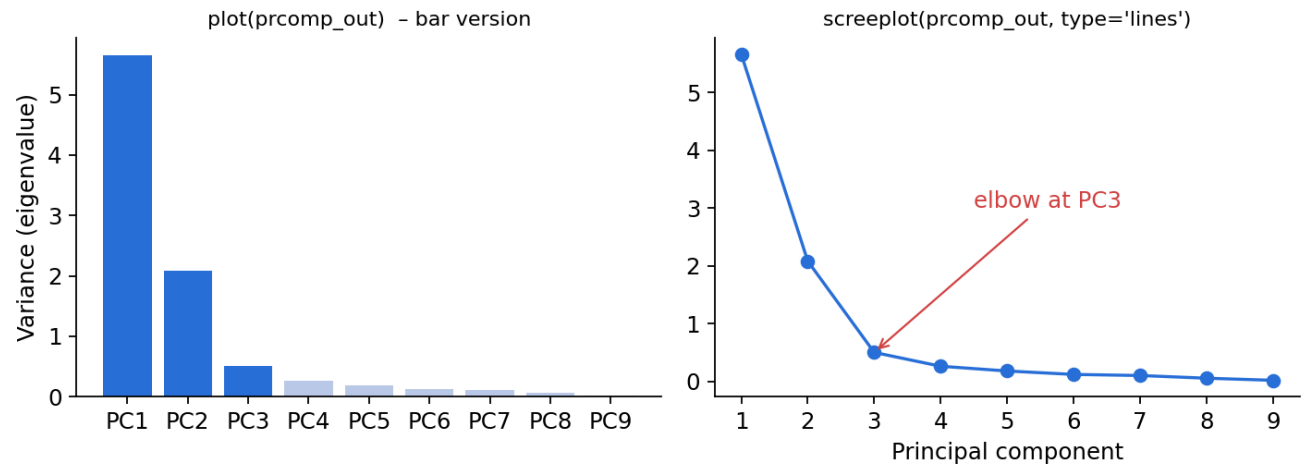
Proportion of variance = (sd of that PC)² ÷ sum of all sd². Since the data are standardized, the total variance = number of variables = 9. For PC1: $2.378^2 = 5.66$, and $5.66 / 9 = 0.628$.



Variance of each PC (bars) and cumulative % (line)

Step 6 – Scree plot and how many PCs

```
#3. scree plot
plot(prcomp_out)          # bar chart of the variances
screeplot(prcomp_out)     # the same plot
# screeplot(prcomp_out, type = "lines") gives the line version
```



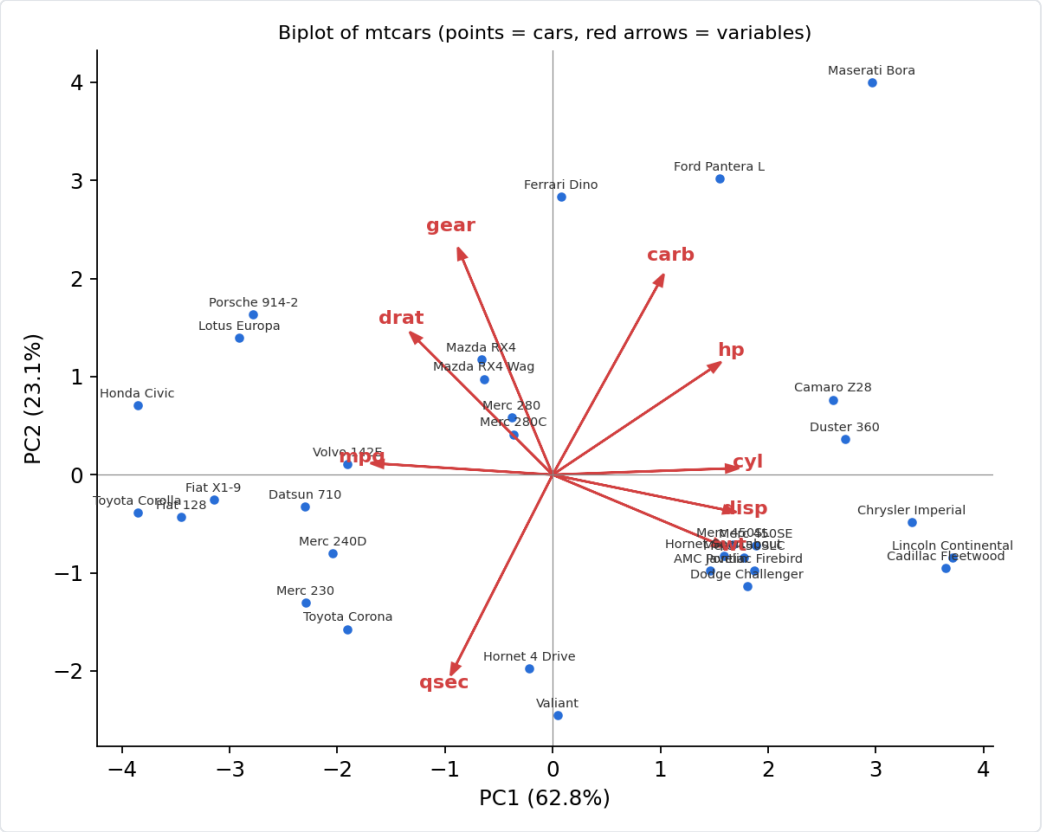
Should we stop at 1, 2, 3 or 4 PCs? It is not easy to decide from the numbers alone, so we look at the scree plot. After PC3 the plot is almost **parallel to the x-axis** (flat). So the **elbow point is 3**.

> **Answer to question 4**
> **3 PCs**, because (a) the scree plot shows the elbow at 3, and (b) the first 3 PCs explain **91.58%** of the variation of the data, which is a good result.
> How much is "enough" depends on the domain and the accuracy needed: some applications need more than 95%, others are fine with 80%.

Step 7 – Visualize with a biplot

```
# visualize
biplot(prcomp_out)
```

`biplot()` plots the **first two PCs**: each car is a point (PC1 on the x-axis, PC2 on the y-axis), and each original variable is a red arrow. We can see clusters, and extra details: variables pointing in the same direction are related.



Heavy, powerful cars (Cadillac, Lincoln) are on the right; small economical cars (Toyota Corolla, Honda Civic) on the left. Your R plot may be mirrored – that is fine (see note in Section 8).

8. Writing down a principal component

A PC is a linear combination of the original (standardized) variables:

$$\text{PC1} = a_1 \cdot \text{mpg} + a_2 \cdot \text{cyl} + a_3 \cdot \text{disp} + a_4 \cdot \text{hp} + a_5 \cdot \text{drat} + a_6 \cdot \text{wt} + a_7 \cdot \text{qsec} + a_8 \cdot \text{gear} + a_9 \cdot \text{carb}$$

How do we get a_1, a_2, …? Look at what is inside the output object:

```
str(prcomp_out)      # shows the parts: sdev, rotation, center, scale, x
?prcomp              # "Value" section explains each part
```

Part	What it contains
`sdev`	Standard deviations of the PCs (square root of the eigenvalues)
`rotation`	The matrix of variable **loadings** – its columns are the **eigenvectors**. (`princomp` calls this `loadings`.)
`center`, `scale`	The centering and scaling used
`x`	The **transformed data** (PC scores)

```
# initial data
data_quants          # columns: mpg, cyl, disp, ...
# transformed data
prcomp_out$x         # columns: PC1, PC2, PC3, ...
# loadings
prcomp_out$rotation
```

In the original data, the first variable was mpg and the second was cyl. In the transformed data, the first variable is **PC1** and the second is **PC2**. Each PC is a mix of all the original variables.

> **The soup (recipe) analogy**
> A PC is like a soup recipe: "this much carrot + this much leeks + this much protein + this much salt + this much spice...". The loadings tell you how much of each variable goes into the PC.

The loadings (correct, 9 variables)

Variable	PC1	PC2	PC3
mpg	−0.3931	0.0275	−0.2212
cyl	0.4026	0.0157	−0.2523
disp	0.3974	−0.0889	−0.0783
hp	0.3671	0.2694	−0.0172
drat	−0.3118	0.3417	0.1500
wt	0.3735	−0.1719	0.4537
qsec	−0.2244	−0.4840	0.6281
gear	−0.2095	0.5508	0.2066
carb	0.2446	0.4843	0.4641

PC1 (from the R code):

$$\text{PC1} = -0.3931\ \text{mpg} + 0.4026\ \text{cyl} + 0.3974\ \text{disp} + 0.3671\ \text{hp} - 0.3118\ \text{drat} + 0.3735\ \text{wt} - 0.2244\ \text{qsec} - 0.2095\ \text{gear} + 0.2446\ \text{carb}$$

Meaning: PC1 is large for cars with many cylinders, big engines, high horsepower and heavy weight, and small for cars with high mpg. So PC1 is like a **"big, powerful, fuel-hungry car"** score.

Homework answer: PC2 and PC3

$$\text{PC2} = 0.0275\ \text{mpg} + 0.0157\ \text{cyl} - 0.0889\ \text{disp} + 0.2694\ \text{hp} + 0.3417\ \text{drat} - 0.1719\ \text{wt} - 0.4840\ \text{qsec} + 0.5508\ \text{gear} + 0.4843\ \text{carb}$$

$$\text{PC3} = -0.2212\ \text{mpg} - 0.2523\ \text{cyl} - 0.0783\ \text{disp} - 0.0172\ \text{hp} + 0.1500\ \text{drat} + 0.4537\ \text{wt} + 0.6281\ \text{qsec} + 0.2066\ \text{gear} + 0.4641\ \text{carb}$$

Note on signs
R may show the **opposite sign** for all loadings of a PC (e.g. every PC2 value multiplied by −1). That is fine: a line pointing "forward" or "backward" is the same line. Always copy the numbers from your own R output.

Calculating a PC score by hand (first car, Mazda RX4)

```
data_standadized[1,]    # first record of the (standardized) dataset
prcomp_out$x[1,1]       # PC1 score of the first car
```

Standardized values of Mazda RX4: mpg 0.1509, cyl −0.1050, disp −0.5706, hp −0.5351, drat 0.5675, wt −0.6104, qsec −0.7772, gear 0.4236, carb 0.7352.

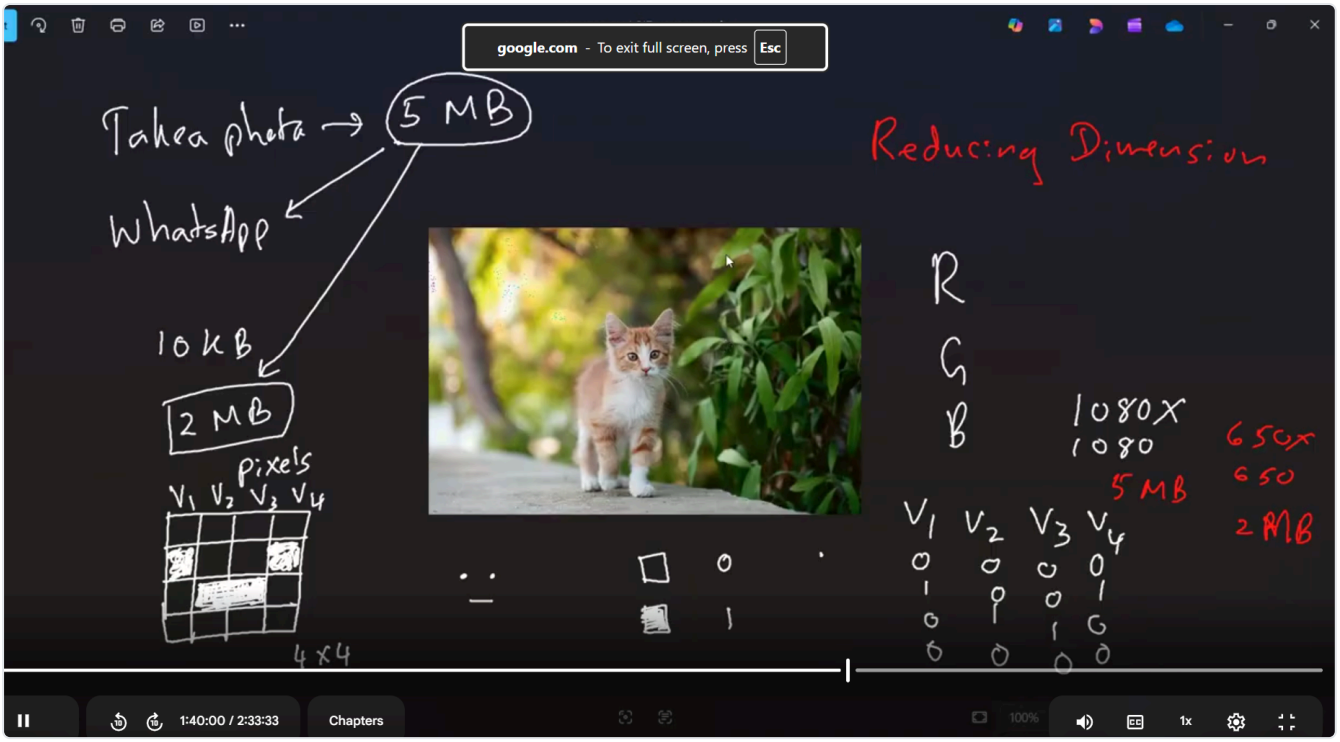
Variable	Loading × standardized value	Result
mpg	−0.3931 × 0.1509	−0.0593
cyl	0.4026 × −0.1050	−0.0423
disp	0.3974 × −0.5706	−0.2267
hp	0.3671 × −0.5351	−0.1964
drat	−0.3118 × 0.5675	−0.1770
wt	0.3735 × −0.6104	−0.2280
qsec	−0.2244 × −0.7772	+0.1744
gear	−0.2095 × 0.4236	−0.0887
carb	0.2446 × 0.7352	+0.1798
Sum = PC1 score		**−0.664**

This matches `prcomp_out$x[1,1]`. So the transformed data is just "loadings × standardized data".

9. PCA is an intermediate step

When we have many variables that are hard to work with, we reduce them to a few PCs. Then we use the **selected PCs** for the **next steps**, for example **regression** or **clustering**.

10. Application: image compression (the cat example)

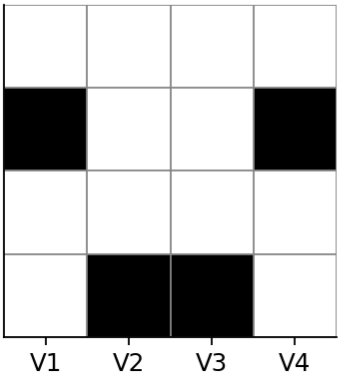


Whiteboard: photo size, pixels, RGB channels and reducing dimension

10.1 The idea

- You take a photo on your phone: **5 MB**. You send it through **WhatsApp**. Your friend downloads it and it is maybe **2 MB** or even **10 KB**. The compression in messaging apps is very good; people even send photos/videos to themselves on WhatsApp to save space.
- An image is made of **pixels**. Suppose a black-and-white image has a **4 × 4 resolution**. Each pixel is either empty (0) or coloured (1). So a smiley face can be stored as numbers. Each column is a variable V1, V2, V3, V4.

4 × 4 pixel smiley face



Stored as numbers
(1 = coloured, 0 = empty)

V1	V2	V3	V4
0	0	0	0
1	0	0	1
0	0	0	0
0	1	1	0

- A colour image has three **channels: Red, Green, Blue (RGB)**. Each channel is a matrix of numbers.
- Resolution such as **1080 × 1080** pixels (e.g. 5 MB). Compressing it to **650 × 650** gives maybe 2 MB. Compression = **reducing dimensions** = reducing space, **without harming the image** (we must not lose valuable information).

- This is a direct application of PCA: keep only the first few PCs of the image.

10.2 Result: more PCs → clearer image



Reconstructing the cat with only the first k PCs of each colour channel (made from the lecture image)

With very few PCs we cannot see a cat. As we add more PCs (the next 10%, 20%, 30% ...) the image becomes clearer. With about 30 PCs the cat is clear but with some noise (grainy areas); with many PCs it looks almost like the original. So we reduce the variables but **keep the information**: we draw a line on the variation explained and keep the PCs above it.

10.3 The R code, line by line

```
# PCA cat example
setwd("C:\\Users\\samee\\My Drive\\Teaching\\ST 4035 - Data Science\\2024\\PCA")
library(jpeg)
cat <- readJPEG('cat.jpeg')
ncol(cat)       # 600 columns
nrow(cat)       # 398 rows
dim(cat)        # 398 600 3  -> a 3-dimensional array
```

- `setwd()` sets the working directory (the folder where the image is). Change this path to **your own folder**.
- `library(jpeg)` loads the jpeg package (`install.packages("jpeg")` the first time).
- `readJPEG()` converts the image into numbers (like the smiley face). The result is a **3-D array**: 398 rows × 600 columns × 3 layers (R, G, B). Values are between 0 and 1.

```
r <- cat[,,1]      # red channel   (398 x 600 matrix)
g <- cat[,,2]      # green channel
b <- cat[,,3]      # blue channel
dim(r)

cat.r.pca <- prcomp(r, center = FALSE)
cat.g.pca <- prcomp(g, center = FALSE)
cat.b.pca <- prcomp(b, center = FALSE)

rgb.pca <- list(cat.r.pca, cat.g.pca, cat.b.pca)
```

- `cat[,,1]` = all rows, all columns, layer 1 → the red matrix. Similarly green and blue.
- PCA is done **separately for each channel**. Here the data are **not standardized** (`center = FALSE`) – this example was borrowed from a book. Each channel has 600 columns (variables).
- The three results are put into one `list`.

```
# first 3
for (i in seq.int(3, round(nrow(cat) - 10), length.out = 10)) {
  pca.img <- sapply(rgb.pca, function(j) {
    compressed.img <- j$x[,1:i] %*% t(j$rotation[,1:i])
  }, simplify = 'array')
  writeJPEG(pca.img, paste('cat_compressed_', round(i,0), '_components.jpg', sep = ''))
}
```

- `seq.int(3, 388, length.out = 10)` gives 10 numbers from 3 to 388: **3, 46, 89, 131, 174, 217, 260, 302, 345, 388**. These are the numbers of PCs to keep.
- For each i, and for each channel j: `j$x[,1:i] %*% t(j$rotation[,1:i])` = (first i PC scores) × (transpose of first i loadings). This **rebuilds** the channel using only i PCs.
- `sapply(..., simplify = 'array')` puts the 3 rebuilt channels back into a 3-D colour image.
- `writeJPEG()` saves it as e.g. `cat_compressed_46_components.jpg`.

```
original <- file.info('cat.jpeg')$size / 1000     # size in KB
imgs <- dir('compressed/')

for (i in imgs) {
  full.path <- paste('compressed/', i, sep='')
  print(paste(i, ' size: ', file.info(full.path)$size / 1000,
              ' original: ', original, ' % diff: ',
              round((file.info(full.path)$size / 1000 - original) / original, 2) * 100,
              '%', sep = ''))
}
```

- This compares each compressed file size with the original (in KB) and prints the % difference.
- In class: with 3 PCs ≈ **16.7 KB**, more PCs ≈ 28.4 KB, with 388 PCs ≈ **30 KB**; the original was ≈ **50 KB**. More PCs → bigger file, clearer picture.

Practical tips for running this code

- The loop saves the files in the working directory, but the size check reads from a folder called `compressed/`. Either move the files into that folder, or save them there directly: `paste('compressed/cat_compressed_', ...)` (create the folder first).
- Rebuilt values can go slightly below 0 or above 1. If `writeJPEG` complains, clip them: `pca.img[pca.img < 0] <- 0; pca.img[pca.img > 1] <- 1`.

Student question: "Why doesn't the resolution of the result image decrease?"

The number of pixels stays the same (398 × 600), but the **amount of information** is reduced. You can see the difference: with few PCs the cat cannot be seen, with more PCs it is clear but noisy, and the file sizes are different (16.7 KB, 28.4 KB, 30 KB vs 50 KB).

11. The theory of PCA – a graphical explanation

These slides were made for non-mathematics (biology / bioinformatics) students, based on the **StatQuest** YouTube video on PCA. They explain the maths with pictures.

11.1 The data

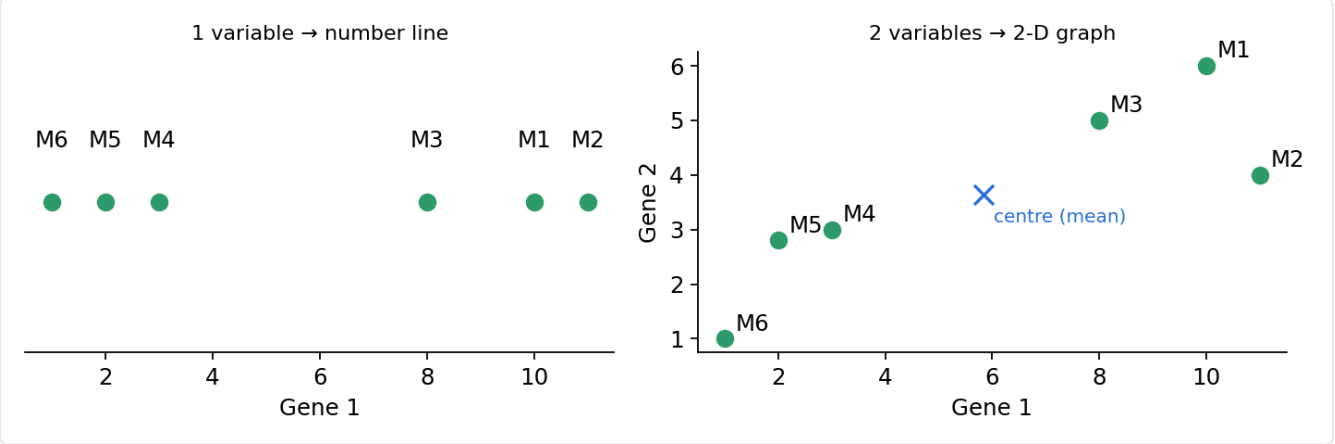
	Mouse 1	Mouse 2	Mouse 3	Mouse 4	Mouse 5	Mouse 6
Gene 1	10	11	8	3	2	1
Gene 2	6	4	5	3	2.8	1

Let's start with a simple data set.

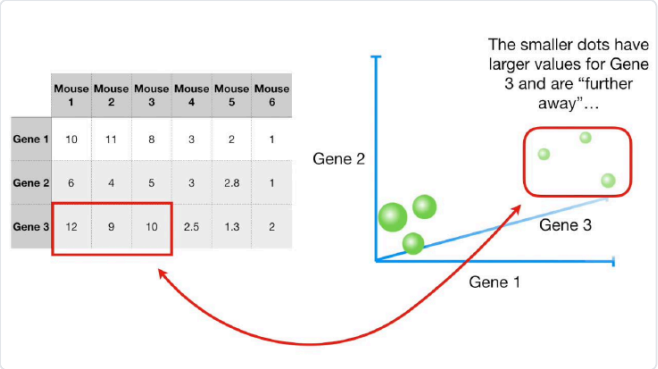
Gene expression of 2 genes in 6 mice

- In bioinformatics we measure **gene expression levels** (like measuring height or weight). Here we measured 2 genes in 6 mice.
- Note: here the **variables are rows** and the **samples (mice) are columns** – this is common in bioinformatics.
- Another example: samples could be high school students, and variables could be test scores (e.g. student 1: maths 95, reading 96; student 2: maths 88, reading 79 …).

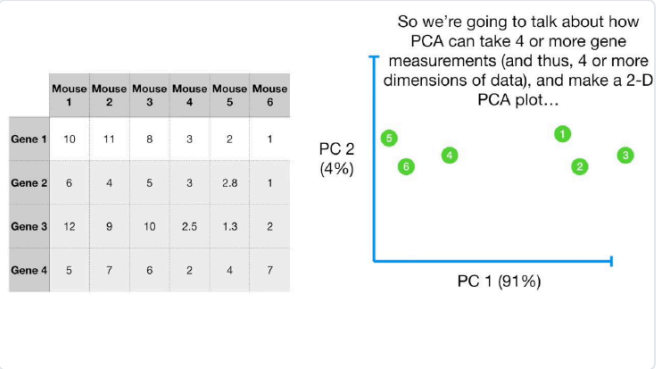
11.2 One, two, three … many variables



- **1 gene** → plot on a **number line**. Easy to see high and low values.
- **2 genes** → a **2-D graph** (Gene 1 on x, Gene 2 on y). We can see patterns: mice 1, 2, 3 are together, mice 4, 5, 6 are together.
- **3 genes** → a 3-D graph (or show the 3rd variable as the **size** of the bubble).
- **4 or more genes** → we **cannot** visualize. We need to summarize the data → **reduce dimensions** with PCA.



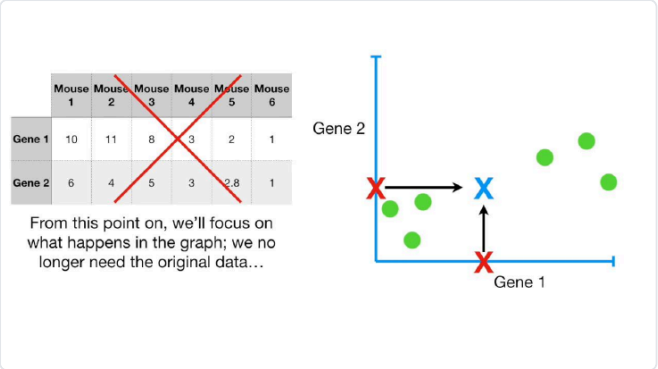
3rd gene shown as a new axis



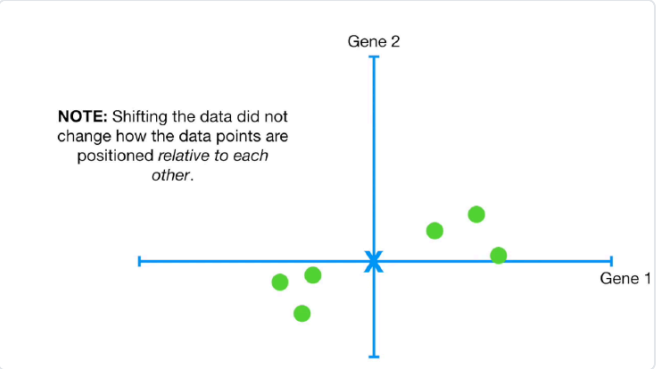
After PCA: a 2-D PCA plot

After PCA we plot PC1 and PC2. For example, PC1 explains 91% and PC2 explains 4% → together 95%, which is fair enough. PCA can also tell us **which variable** (e.g. Gene 3) is most responsible for separating the clusters.

11.3 Step 1 – Centre the data



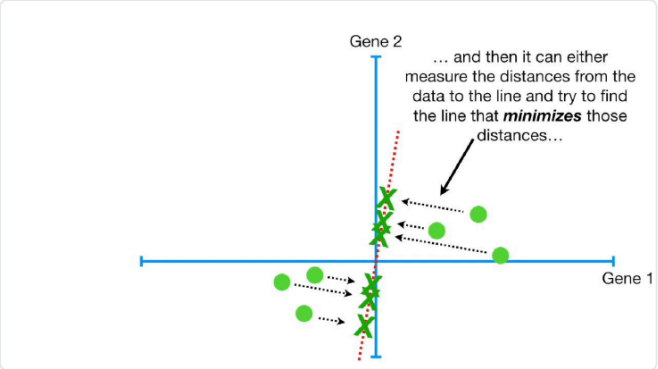
Average of Gene 1 and Gene 2 gives the centre



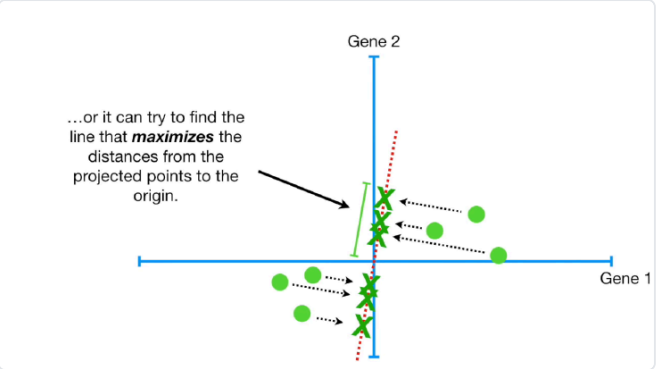
Shift the data so the centre is at the origin

- Find the **mean of each variable** (project the points onto each axis and take the average). With the data above: mean Gene 1 = 35/6 = 5.83, mean Gene 2 = 21.8/6 = 3.63.
- The point (mean of Gene 1, mean of Gene 2) is the **centre** of the data.
- **Shift** the data so the centre is at the **origin (0, 0)**. In practice: **subtract the mean** from each data point (e.g. mouse 1: 10 − 5.83 = 4.17, 6 − 3.63 = 2.37).
- Shifting does **not** change how the points are positioned relative to each other – the same gaps are kept. Only the origin changes.

11.4 Step 2 – Fit the best line through the origin



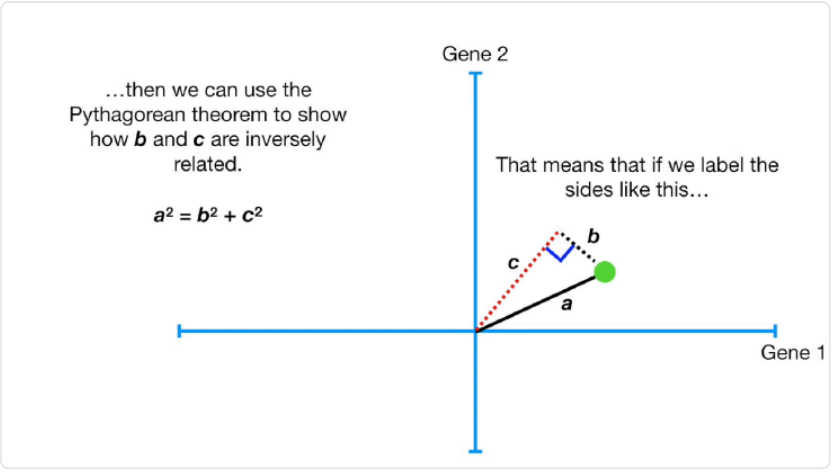
Minimize the distance from the points to the line…



…or maximize the distance from the projected points to the origin

- Draw a random line through the origin, then **rotate** it until it fits the data best.
- The "best" line has the **smallest deviations** from the points to the line.
- **Projection:** drop each point onto the line at a right angle. The point where it lands is the projection.
- As the line fits better, the distance from each point to the line gets **smaller**, and the distance from the projected point to the origin gets **larger**.

11.5 Why? – Pythagoras

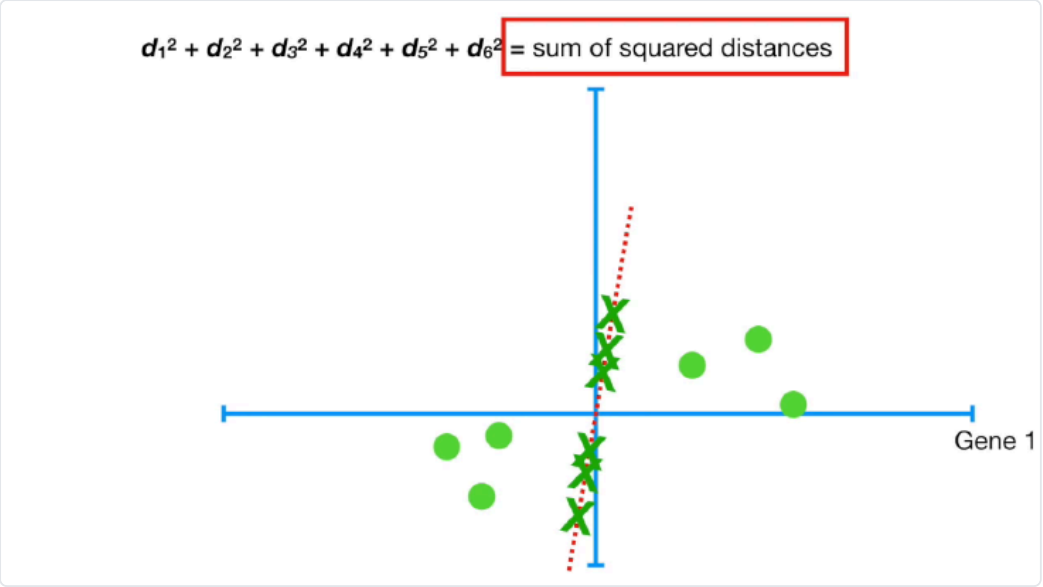


Look at one point. The distance from the origin to the point (a) does **not** change when the line rotates. The projection makes a **right angle**, so by Pythagoras:

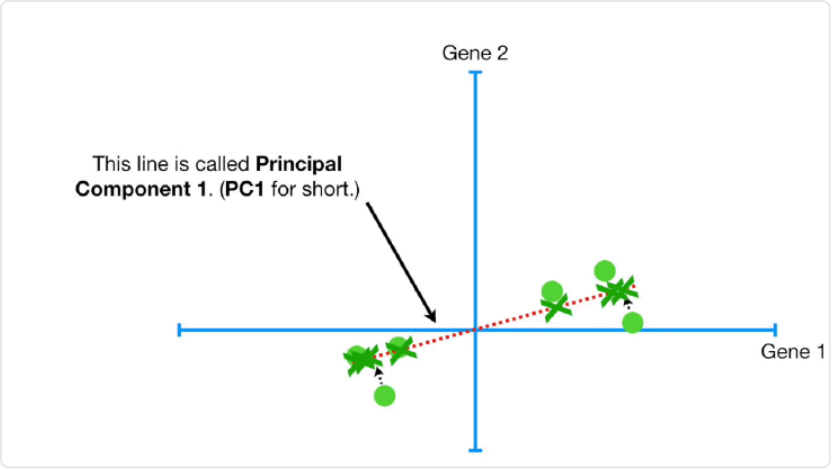
$$a^2 = b^2 + c^2$$

where b = distance from the projected point to the origin, and c = distance from the point to the line. Since a^2 is constant: **if b gets bigger, c must get smaller**, and vice versa. So PCA can either **minimize c** (distance to the line) or **maximize b** (distance from the projected point to the origin). They are the same thing. PCA maximizes b because it is easier to calculate.

11.6 Sum of squared distances

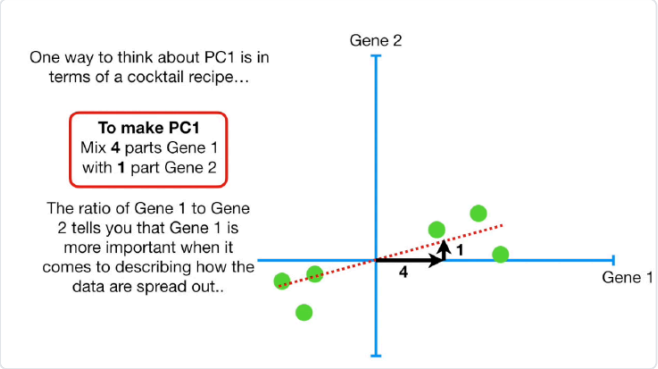


- Measure the distance from each projected point to the origin: d1, d2, ..., d6.
- Some are on the positive side and some on the negative side, so they would cancel. We **square** them so negatives don't cancel positives.
- Add them: $d1^2 + d2^2 + ... + d6^2$ = **sum of squared distances, SS(distances)**.
- Rotate the line, project, compute SS again ... until we get the **largest SS(distances)**.

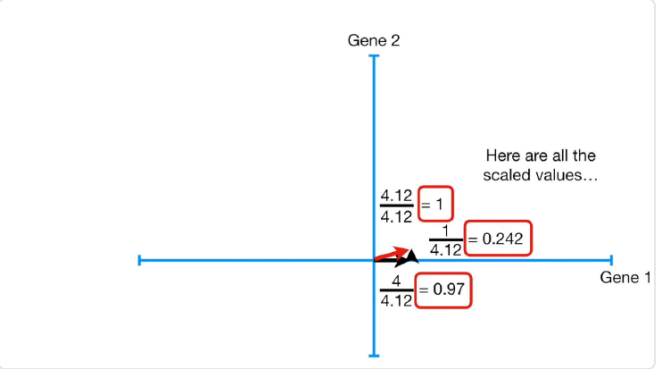


The line with the largest SS(distances) is Principal Component 1 (PC1)

11.7 PC1 as a recipe – loading scores

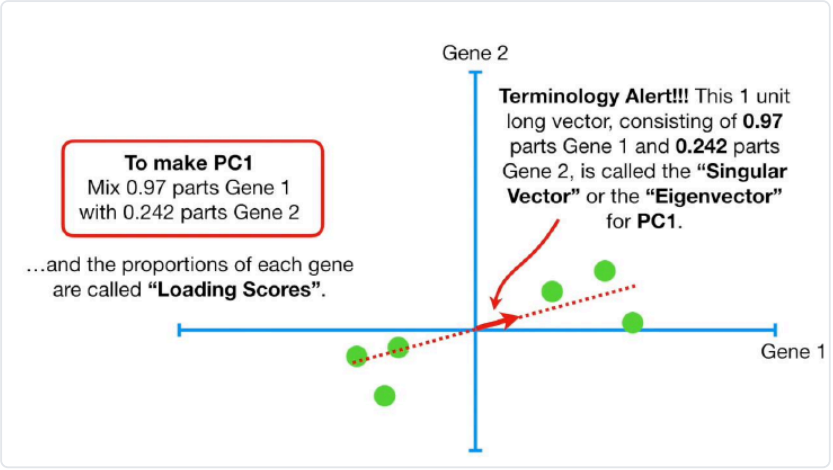


Slope 0.25: go 4 along Gene 1, 1 up along Gene 2



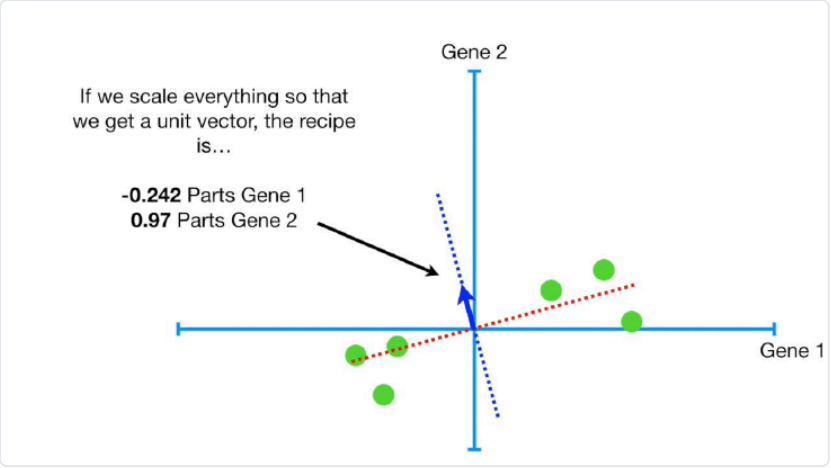
Scale to a unit vector: divide by 4.12

- A line through the origin is defined by its **slope** only (intercept is 0). In the slide example, PC1 has **slope 0.25**: for every **4 units along Gene 1**, go **1 unit up** along Gene 2 (1/4 = 0.25 = tan θ).
- This means the data are spread out mostly along Gene 1 and only a little along Gene 2.
- **Cocktail recipe:** "To make PC1, mix **4 parts Gene 1** with **1 part Gene 2**." Gene 1 is more important for describing how the data spread.
- Mathematically, PC1 is a **linear combination** of Gene 1 and Gene 2.
- **Make it a unit vector** (length 1): the long side = $\sqrt{(4^2 + 1^2)} = \sqrt{17} = \mathbf{4.12}$. Divide every side by 4.12: 4/4.12 = **0.97**, 1/4.12 = **0.242**, 4.12/4.12 = 1.
- New recipe: **0.97 parts Gene 1 + 0.242 parts Gene 2**. Same direction, just standardized.



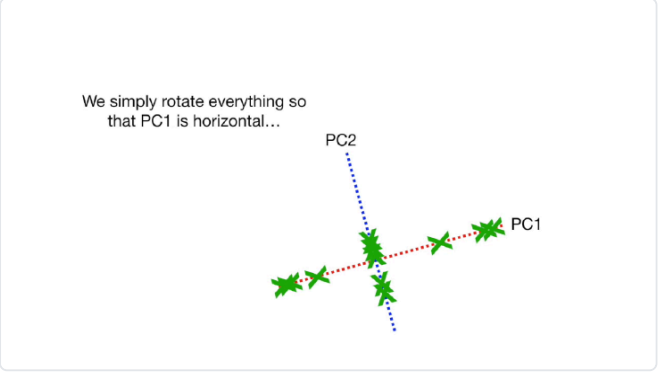
Term	Meaning
Eigenvector (singular vector) for PC1	The unit vector (0.97, 0.242)
Loading scores	The numbers 0.97 and 0.242 (the proportions of each variable). In R: `$rotation`
Eigenvalue for PC1	SS(distances) for PC1
Singular value for PC1	√(eigenvalue for PC1)

11.8 PC2

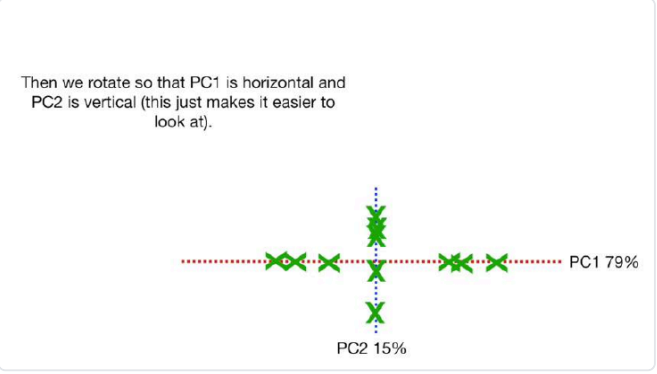


- PCs are **orthogonal** = **perpendicular** to each other (in statistics: **independent** / uncorrelated).
- In higher dimensions there are **infinitely many** lines perpendicular to PC1 (you can rotate around PC1). PC2 is the perpendicular line with the **second highest SS(distances)**.
- With only 2 variables, PC2 is simply the line through the origin perpendicular to PC1 – no further optimization needed.
- Recipe for PC2: **−1 part Gene 1 + 4 parts Gene 2**. As a unit vector: **−0.242 Gene 1 + 0.97 Gene 2**. These are the loading scores for PC2 (Gene 2 is 4 times more important than Gene 1 for PC2).
- Eigenvalue for PC2 = SS(distances) for PC2.

11.9 Final PCA plot



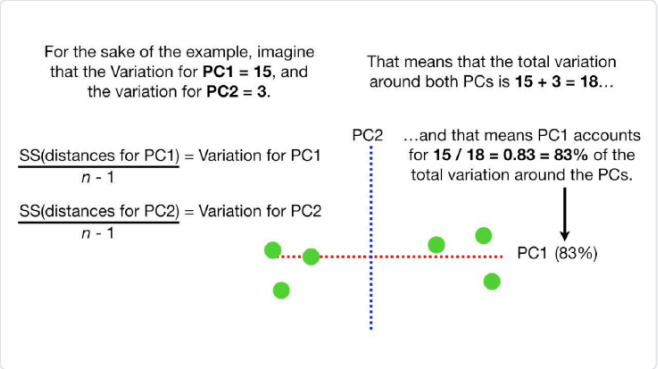
Rotate so PC1 is horizontal

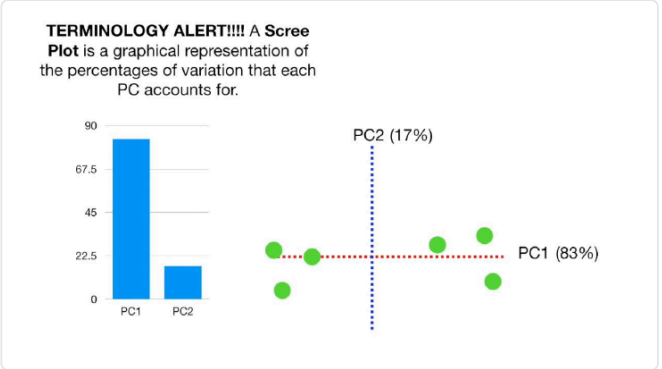


Use the projected points to place each sample

Rotate everything so **PC1 is the x-axis** and **PC2 is the y-axis**. Each sample is placed using its projected points on PC1 and PC2. These are the new transformed variables.

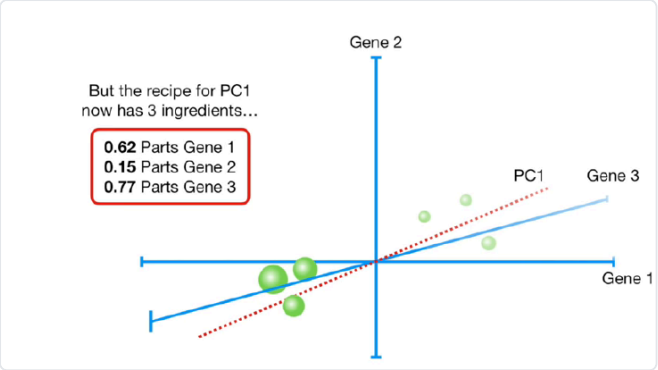
11.10 Variation explained and the scree plot



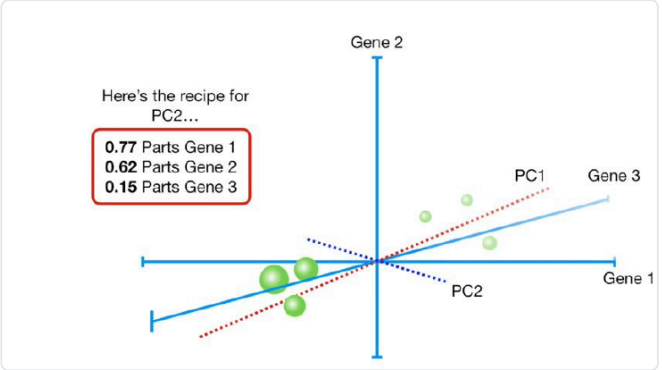


- Convert SS(distances) to **variation** by dividing by the sample size minus 1: **Variation = SS(distances) / (n − 1)**. This is the same as the sample variance formula $\Sigma(x_i - \bar{x})^2 / (n - 1)$.
- Example: variation PC1 = 15, variation PC2 = 3 → total = 18.
- PC1 explains 15/18 = **83%**, PC2 explains 3/18 = **17%**.
- A **scree plot** is a graph of the percentages of variation that each PC accounts for.

12. PCA with 3 or more variables, and when PCA does not work

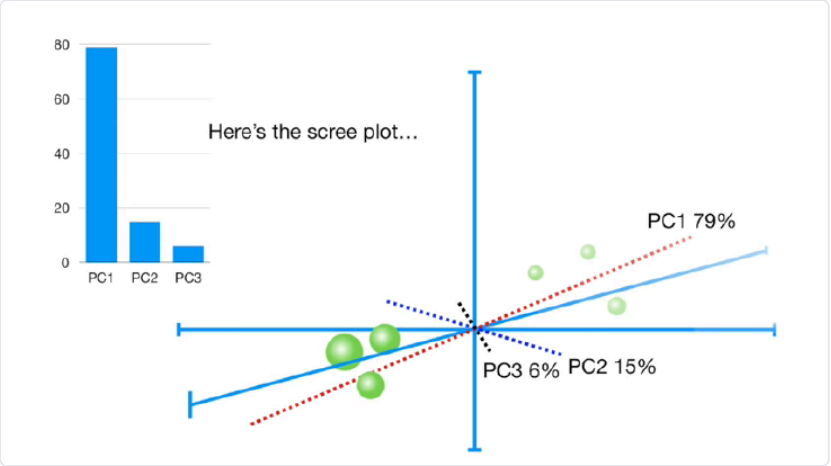


PC1 recipe with 3 genes



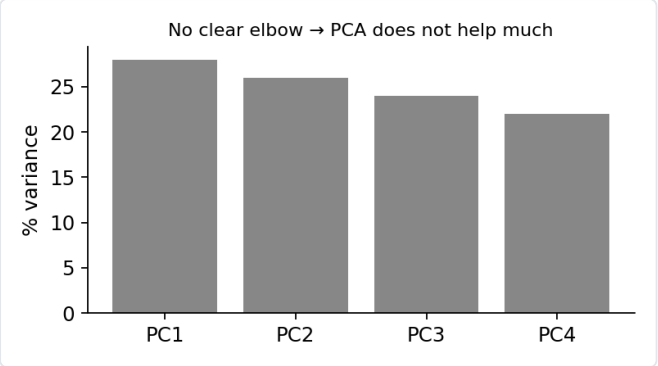
PC2 recipe

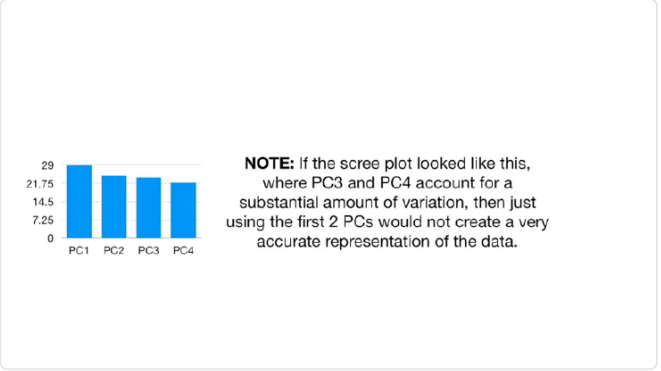
- With 3 genes we do the same: centre the data, find the best-fitting line through the origin (PC1), then PC2 perpendicular to PC1, then PC3 perpendicular to both.
- PC1 recipe: 0.62 Gene 1 + 0.15 Gene 2 + **0.77 Gene 3** → Gene 3 is the most important for PC1.
- PC2 recipe: **0.77 Gene 1** + 0.62 Gene 2 + 0.15 Gene 3 → Gene 1 is the most important for PC2.
- In theory there is one PC per variable (or per sample, whichever is smaller).
- Here PC1 = 79%, PC2 = 15%, PC3 = 6%. So PC1 + PC2 = **94%**. A 2-D plot of PC1 and PC2 is a good approximation of the 3-D graph. We lose only 6% but we gain a simple 2-D picture – a fair trade.



- With **4 genes** we cannot draw a 4-D graph, but PCA still works: e.g. PC1 + PC2 explain 90% → plot them in 2-D.

When PCA does not help





If the scree plot has **no clear elbow** – all PCs have about the same height (e.g. PC3 and PC4 also explain a large amount) – then using only the first two PCs would **not** give an accurate picture. If we drop PCs, the **information loss is significant**. However, even a noisy PCA plot can still be used to identify clusters.

> **Next week**
> What if **all** variables are **categorical**? Then PCA cannot be used (we would have to remove everything). Next week: **other dimension reduction techniques** for such data.

Reference: StatQuest, "Principal Component Analysis (PCA), Step-by-Step" – **youtu.be/FgakZw6K1QQ**. StatQuest has very clear explanations of most statistics and data science topics.

13. Summary, homework and revision questions

Key points

- Dimension = number of variables. Dimension reduction creates a **smaller set of new variables** that keeps most of the information.
- PCs are **linear combinations** of all original variables; PC1 explains the most variation, PC2 the next, etc. PCs are **orthogonal** (uncorrelated).
- PCA: **quantitative data only** → **standardize** → covariance matrix → eigenvalues/eigenvectors → loadings → transformed data → choose number of PCs.
- Choose PCs with the **scree (elbow) plot** and the **cumulative % variance** (80%–95% depending on the field).
- R: `prcomp(x, center = TRUE, scale. = TRUE)`, `summary()`, `plot()` / `screeplot()`, `biplot()`, `$rotation` = loadings, `$x` = scores.
- mtcars: remove vs and am, standardize → **3 PCs** explain **91.58%**.
- Geometry: centre the data, find the line that **maximizes SS(distances)** of projected points (= minimizes distances to the line, by Pythagoras). Unit vector = eigenvector; SS = eigenvalue; SS/(n−1) = variance.
- PCA is an **intermediate step** (then regression, clustering…). Application: **image compression**.

Homework

Write down PC2 and PC3 for mtcars (answers in Section 8 – check them against your own R output).

Test yourself

1. What does "dimension" mean in a data set? Does dimension reduction delete variables?
2. Write the general form of the first principal component for 100 variables.
3. List the steps of PCA.
4. Why must we standardize before PCA? What happens in mtcars if we don't?
5. Why can't we use `sapply(data, is.numeric)` to remove vs and am?
6. Which two R functions do PCA, and which package are they in?
7. PC1 = 50%, PC2 = 30%, PC3 = 12%, PC4 = 5%. Find the cumulative % for the first 3 PCs. (Answer: 92%)
8. What is an elbow point? What does it mean if the scree plot has no elbow?
9. In `prcomp` output, where are the loadings and where is the transformed data?
10. A slope of PC1 is 0.25. Find the unit-vector loading scores. (Answer: 0.97 and 0.242)
11. Explain with Pythagoras why minimizing the distance to the line is the same as maximizing the distance from the projected points to the origin.
12. Variation PC1 = 12, PC2 = 4. What % does each explain? (Answer: 75% and 25%)
13. Explain how PCA can be used to compress an image.